

Getting Started with the Raspberry Pi Canva Kiosk

Teofil Wetzel

February 12, 2025

Contents

1 Introduction

This document provides a brief guide on how to set up a Raspberry Pi so that it automatically launches a Canva presentation in fullscreen (kiosk) mode on boot. We will use:

- **Chromium** as the browser
- **xdotool** for automated mouse and keyboard events
- **systemd** to start our kiosk script at boot time

2 Prerequisites

- A Raspberry Pi with a recent version of Raspberry Pi OS installed
- An internet connection (wired or Wi-Fi)
- An HDMI display, plus keyboard & mouse for the initial configuration

3 System Preparation

3.1 Update and Upgrade

Open a terminal and run:

```
sudo apt-get update
sudo apt-get upgrade -y
```

Afterwards, reboot if necessary:

```
sudo reboot
```

3.2 Install Required Packages

We need Chromium and xdotool:

```
sudo apt-get install -y chromium-browser xdotool
```

4 Configuring the Kiosk Script

4.1 Create a Script

Create the file `/home/pi/kiosk_start.sh` (adjust the path and username if needed). Below is an example that reads a Canva URL from an external text file called `canva_url.txt`. Make sure to update coordinates and click positions if necessary.

```

#!/bin/bash
# This script starts Chromium in kiosk mode, waits for the visible window,
# then sends Ctrl+Alt+P to enter fullscreen, and simulates mouse clicks
# to enable autoplay and close the menu in Canva.

DEBUG=0
[ "$DEBUG" = "1" ] && set -x

URL_FILE="/home/pi/canva_url.txt"

DISPLAY_NUM=":0"
TIMEOUT=20
SLEEP_INTERVAL=2
KEY_SEND_DELAY=25
CLICK_DELAY=5

log() {
    echo "[$(date +%H:%M:%S)] $*"
}

# 1) Read URL from external file
if [ ! -f "$URL_FILE" ]; then
    echo "Error: URL file not found at $URL_FILE. Exiting."
    exit 1
fi

URL="$(cat "$URL_FILE" | tr -d '[:space:]')"
if [ -z "$URL" ]; then
    echo "Error: The URL in $URL_FILE is empty. Exiting."
    exit 1
fi

# 2) Terminate existing Chromium instances
if pgrep -x "chromium" > /dev/null; then
    log "Chromium is already running. Terminating old instance..."
    pkill -x chromium
    sleep 2
fi

# 3) Wait for the X server
log "Waiting for the X server..."
export DISPLAY="$DISPLAY_NUM"
export XAUTHORITY="/home/pi/.Xauthority"
sleep 5

```

```

# 4) Launch Chromium in kiosk mode
log "Starting Chromium in kiosk mode with URL: $URL"
chromium-browser --kiosk --noerrdialogs --disable-infobars \
    --ozone-platform=x11 "$URL" > /dev/null 2>&1 &
sleep 2

# 5) Wait for the Chromium window
elapsed=0
CHROMIUM_WINDOW_ID=""
log "Looking for the Chromium window..."
while [ -z "$CHROMIUM_WINDOW_ID" ] && [ "$elapsed" -lt "$TIMEOUT" ]; do
    sleep "$SLEEP_INTERVAL"
    elapsed=$((elapsed + SLEEP_INTERVAL))
    CHROMIUM_WINDOW_ID=$(xdotool search --onlyvisible --class "chromium" \
        2>/dev/null | head -n 1)
    log "Searching for Chromium window... ($elapsed seconds)"
done

if [ -z "$CHROMIUM_WINDOW_ID" ]; then
    log "Error: No Chromium window found! Exiting."
    exit 1
fi

# 6) Activate the window
log "Chromium window found: $CHROMIUM_WINDOW_ID. Activating window..."
xdotool windowactivate --sync "$CHROMIUM_WINDOW_ID"
sleep 1

# 7) Wait to allow the page to load
log "Waiting $KEY_SEND_DELAY seconds for the page to fully load..."
sleep "$KEY_SEND_DELAY"

# 8) Send Ctrl+Alt+P to enter fullscreen (Canva)
log "Sending Ctrl+Alt+P shortcut..."
xdotool key --clearmodifiers --delay 100 ctrl+alt+p
if [ $? -ne 0 ]; then
    log "Error: Could not send the keyboard shortcut."
else
    log "Fullscreen shortcut sent successfully!"
fi

# 9) Wait before mouse actions
log "Waiting $CLICK_DELAY seconds before mouse actions..."

```

```

sleep "$CLICK_DELAY"

# 10) Simulate mouse movement and clicks
log "Moving mouse to (1820,1050) to display the menu..."
xdotool mousemove 1820 1050
sleep 1
log "Clicking on the menu area..."
xdotool click 1
sleep 1

log "Moving mouse to (1559,872) to click the autoplay button..."
xdotool mousemove 1559 872
sleep 1
xdotool click 1
log "Autoplay enabled."
sleep 1

# 11) Close the menu with a click at (0,0)
log "Clicking on (0,0) to close the menu..."
xdotool mousemove 0 0
sleep 1
xdotool click 1
log "Menu closed."

sleep 2
log "Kiosk script finished."

```

Make the script executable:

```
chmod +x /home/pi/kiosk_start.sh
```

4.2 Create the URL File

Place your Canva URL in a text file:

```
echo "https://www.canva.com/design/EXAMPLE_LINK" > /home/pi/canva_url.txt
chmod 644 /home/pi/canva_url.txt
```

5 Running the Script Automatically on Boot

5.1 Create a systemd Service

Make a file `/etc/systemd/system/start_canva.service` with the following content:

```
[Unit]
Description=Canva Kiosk Service
After=display-manager.service
Requires=display-manager.service

[Service]
User=pi
Environment=DISPLAY=:0
Environment=XAUTHORITY=/home/pi/.Xauthority
WorkingDirectory=/home/pi
ExecStart=/home/pi/kiosk_start.sh
Restart=on-failure
RestartSec=10
KillMode=none
RemainAfterExit=yes
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=graphical.target
```

Adjust User, WorkingDirectory, and file paths if needed.

5.2 Enable and Start the Service

1. Reload systemd:

```
sudo systemctl daemon-reload
```

2. Enable the service:

```
sudo systemctl enable start_canva.service
```

3. Start the service immediately for testing:

```
sudo systemctl start start_canva.service
```

4. Check status and logs:

```
sudo systemctl status start_canva.service
sudo journalctl -u start_canva.service
```

6 Conclusion

With this setup, your Raspberry Pi will automatically load a Canva presentation in fullscreen (kiosk) mode at boot. The script also simulates clicks to enable autoplay and closes the menu, providing a near-hands-free presentation experience. Adjust the mouse coordinates and waiting times as needed based on your screen resolution and the layout of Canva's UI elements.